Programming Techniques for Supercomputers - SS 2025

2D Modelling of the Steady State Heat Equation

Aman Sayyad	Muhammad Nabeel	Bilal Bostan
ys42ibuq	ry25vufe	ri93vity
23461898	23591108	23233769
aman.sayyad@fau.de	muhammad.nabeel@fau.de	bilal.bostan@fau.de

May 1, 2026

Contents

1 *PART 1* **3**
- 1.1 Layer Condition . . . 3
- 1.2 General *Roofline* model . . . 3
- 1.3 Kernels . . . 5
  - 1.3.1 (K1) applyStencil(lhs,x): . . . 5
  - 1.3.2 (K2) axpby(lhs[:]= a*x[:]+b*y[:]): . . . 7
  - 1.3.3 (K3) dotProduct: . . . 8
  - 1.3.4 (K4) Gs PreCon: . . . 10
- 1.4 Formulae . . . 13
- 1.5 *roofline* model for CG . . . 13
  - 1.5.1 for grid size 2000x20000 (Ny x Nx) . . . 14
  - 1.5.2 for grid size 20000 × 2000 . . . 14
  - 1.5.3 for grid size 1000x40000 . . . 15
- 1.6 *roofline* model for PCG . . . 16
  - 1.6.1 for grid size 2000 * 20000 . . . 17
  - 1.6.2 for grid size 20000 * 2000 . . . 18
  - 1.6.3 for grid size 1000 * 400000 . . . 19
- 1.7 Conclusion . . . 20

2 Part 2 **21**
- 2.1 ptfscam.sh . . . 22
- 2.2 Performance Results . . . 24

3 Part 3 **25**
- 3.1 for grid size 2000 * 20000 . . . 25
  - 3.1.1 applyStencil . . . 25
  - 3.1.2 GSPrecon . . . 26
- 3.2 for grid size 20000 * 2000 . . . 27
  - 3.2.1 applyStencil . . . 27
  - 3.2.2 GSPrecon . . . 27
- 3.3 for grid size 1000 * 400000 . . . 28
  - 3.3.1 applyStencil . . . 28
  - 3.3.2 GSPrecon . . . 28

4 Part 4 **29**

1. *PART 1*

Calculate roofline predictions in [LUP/s] for CG and PCG on 1 ccNUMA domain (18 cores) of Fritz. Calculate for three grid sizes : 2000 × 20000, 20000 × 2000 and 1000 × 400000. The last dimension is in x-direction (innermost). Does the performance change? Why?

1.1. Layer Condition

As we are working with stencil computations, we first examine whether the working data set fits entirely into the L3 cache. To this end, we use the following *layer condition*:

$$\text{Layer Condition} = xSize \times 8\,\text{B} \times n_{\text{threads}} \times 3 \text{ rows} \;\leq\; \frac{\text{L3 cache size}}{2}.$$

The L3 cache size on the *FRITZ* is 54 MB, and thus the effective cache capacity considered is 27 MB.

- For $xSize = 2000$,

  $$\text{Layer Condition} = 2000 \times 8\,\text{B} \times 18 \times 3 = 0.864\,\text{MB}.$$

  Since $0.864\,\text{MB} \leq 27\,\text{MB}$, the layer condition is satisfied, and the data fits into the L3 cache.

- For $xSize = 20000$,

  $$\text{Layer Condition} = 20000 \times 8\,\text{B} \times 18 \times 3 = 8.64\,\text{MB}.$$

  Since $8.64\,\text{MB} \leq 27\,\text{MB}$, the layer condition is satisfied, and the data fits into the L3 cache.

- For $xSize = 400000$,

  $$\text{Layer Condition} = 400000 \times 8\,\text{B} \times 18 \times 3 = 172.8\,\text{MB}.$$

  Since $172.8\,\text{MB} > 27\,\text{MB}$, the layer condition is not satisfied, and the data does not fit into the L3 cache.

Consequently, we compute the Roofline model predictions for the different kernels, categorizing them according to whether the corresponding $xSize$ satisfies the L3 cache layer condition.

1.2. General *Roofline* model

To assess whether the kernels are memory-bound or compute-bound, we evaluate their performance using the refined *Roofline* model. This model incorporates the applicable peak performance $P_{\max}$, the computational intensity I, defined as the amount of work per byte transferred, and the applicable saturated peak bandwidth b_s.

Once we have such quantities computed, the expected performance given by the model is

$$P = \min\big(P_{\max},\, I \cdot b_s\big). \tag{1}$$

Let us summarize some data of Fritz and some assumptions in order to get an explicit model:

- For the *roofline* modelling we assume the memory bandwidth of 1 ccNUMA domain of Fritz is 82 GB/s.
- Kernels: `applyStencil (K1)`, `AXPBY (K2)`, `dotProduct (K3)`, `GSPreCon (K4)`.
- Assume AVX512 registers.

For an Ice Lake SP core with AVX512 the throughput per cycle is:

- LD/ST throughput per cycle: 2 LD, 1 ST.
- ADD throughput: 2 / cy.
- MUL throughput: 2 / cy.
- FMA throughput: 2 / cy.

The idea is to study one by one the kernels, as general functions of the grid size, their layer conditions, intensities, $P_{\max}$ and all that is necessary for the *roofline* model. Then for each grid size set we will find the exact values for both CG (K1, K2, K3) and PCG (K1, K2, K3, K4) being careful to take them together correctly (e.g., summing up the timings as in the project intro slides, and not by summing up the single performances).

The execution units for **AVX512 for an Intel Ice Lake CPU** look like the following. This will be useful for computing $P_{\max}$.

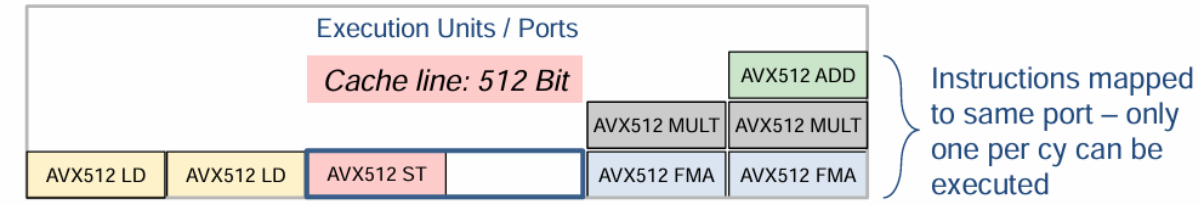


Figure 1: Execution units for AVX512 on Ice Lake CPU (Fritz)

We can therefore do:

- 2 AVX512 LD Instruction per cycle
- 1 AVX512 ST Instruction per cycle
- 2 AVX512 FMA Instruction per cycle

as this core has a maximum 5 inst/cy that can be executed.

1.3. Kernels

1.3.1 (K1) applyStencil(lhs,x):

```
for ( int j=1; j<ySize-1; ++j)
{
    for ( int i=1; i<xSize-1; ++i)
    {
        (*lhs)(j,i) = w_c*(*x)(j,i) - w_y*((*x)(j+1,i) + (*x)(j-1,i))
         - w_x*((*x)(j,i+1) + (*x)(j,i-1));
    }
}
```

1. For xSize=2000, xSize=20000
   Since the data fits perfectly into the L3 cache, we have

   - 2 AVX512 LD
   - 1 AVX512 ST
   - 1 AVX512 ADD
   - 3 AVX512 FMA

   Due to the amount of FMA operations, this instruction set requires 2 cycles with AVX512 registers.

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{cycle} = \frac{56\ FLOPS}{2\ cycles} = 28\frac{FLOPS}{cy}$$

   And therefore,

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 28 \times 18 \times 2.2 = 1108.8\frac{GFLOPs}{s}$$

   Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{7}{3 \times 8} = \frac{7}{24}\frac{FLOPS}{Bytes}$$

   And we know that the bandwidth $b_s = 82GB/s$. Thus, peak performance for the kernel applystencil for is given by

$$\begin{aligned} P_{xSize=2000,20000}^{\text{applyStencil}} &= \min\left\{P_{\text{1 ccNUMA}}^{\max},\ I \times b_s\right\} \\ &= \min\left\{1108.8\ \frac{\text{GFLOPS}}{\text{s}},\ \frac{7 \times 82}{24}\ \frac{\text{GFLOPS}}{\text{s}}\right\} \\ &= 23.92\ \frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{2}$$

2. For xSize=400000
   Since the data does not fit perfectly into the L3 cache, the kernel becomes memory-bound, and we can update the port usage accordingly:

   - 4 AVX512 LD
   - 1 AVX512 ST

- 1 AVX512 ADD
- 3 AVX512 FMA

With larger data, the memory bandwidth limits performance rather than the core peak and the computational intensity remains the same.

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{7}{5 \times 8} = \frac{7}{40} \frac{\text{FLOPS}}{\text{Byte}}$$

The achievable performance is therefore limited by the memory bandwidth $b_s = 82\,\text{GB/s}$:

$$\begin{aligned} P^{\text{applyStencil}}_{xSize=400000} &= \min \left\{ P^{\max}_{\text{core}},\, I \times b_s \right\} \\ &= \min \left\{ 1108.8 \, \frac{\text{GFLOPS}}{\text{s}},\, \frac{7 \times 82}{40} \, \frac{\text{GFLOPS}}{\text{s}} \right\} \\ &= 14.35 \, \frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{3}$$

*1.3.2 (K2) axpby(lhs[:]= a*x[:]+b*y[:]):*

```
for(int yIndex=shift; yIndex<lhs->numGrids_y(true)-shift; ++yIndex)
{
    for(int xIndex=shift; xIndex<lhs->numGrids_x(true)-shift; ++xIndex)
    {
        (*lhs)(yIndex,xIndex) = (a*(*x)(yIndex,xIndex))
                              + (b*(*y)(yIndex,xIndex));
    }
}
```

Since there is no stencil operation here, the following computations hold true for xSize=2000, xSize=20000 and xSize=400000. In the following Kernel We have;

- – 2 LD
  – 1 ST
  – 2 MUL
  – 1 ADD
- This means that we can combine the operations into 1 FMA and 1 MUL using AVX512 Registers.

Therefore, all operations can be carried out in 1 cycle. Thus,

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{cycle} = \frac{24\ FLOPS}{1\ cycles}$$

and therefore

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 16 \times 18 \times 2.2 = 950.4 \frac{GFLOPS}{s}$$

Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{3}{3 \times 8} = \frac{3}{24} \frac{FLOPS}{Bytes}$$

And we know that the bandwidth $b_s = 82GB/s$. Thus, peak performance for the kernel applystencil for is given by

$$\begin{aligned} P^{\text{axpby}} &= \min\left\{P_{\text{1 ccNUMA}}^{\text{max}},\ I \times b_s\right\} \\ &= \min\left\{950.4\,\frac{\text{GFLOPS}}{\text{s}},\ \frac{3 \times 82}{24}\,\frac{\text{GFLOPS}}{\text{s}}\right\} \\ &= 10.25\,\frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{4}$$

1.3.3 (K3) dotProduct:

```
for(int yIndex=shift; yIndex<x->numGrids_y(true)-shift; ++yIndex)
{
    for(int xIndex=shift; xIndex<x->numGrids_x(true)-shift; ++xIndex)
    {
        dot_res += (*x)(yIndex,xIndex)*(*y)(yIndex,xIndex);
    }
}
```

1. For $x \neq y$
   Since there is no stencil operation here, the following computations hold true for xSize=2000, xSize=20000 and xSize=400000. In the following Kernel We have;

   - – 2 LD
     – 1 MUL
     – 1 ADD
   - This means that we can combine the operations into 1 FMA AVX512. Registers.

   Therefore, all operations can be carried out in 1 cycle. Also, note that because we accumulate on dot_res, this variable always stays in the register and it does not count as a ST instruction in the kernel. This is because accumulation happens within the register and no additional memory writes are required until after the loop is completed. Thus,

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{\text{cycle}} = \frac{16\ FLOPS}{1\ cycles}$$

   and therefore

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 16 \times 18 \times 2.2 = 633.6 \frac{GFLOPS}{s}$$

   Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{2}{2 \times 8} = \frac{2}{16} \frac{FLOPS}{Bytes}$$

   And we know that the bandwidth $b_s = 82GB/s$. Thus, peak performance for the kernel dotProduct is given by

$$\begin{aligned} P_{x \neq y}^{\text{dotProduct}} &= \min \left\{ P_{\text{1 ccNUMA}}^{\max}, I \times b_s \right\} \\ &= \min \left\{ 633.6 \, \frac{\text{GFLOPS}}{\text{s}}, \frac{2 \times 82}{16} \, \frac{\text{GFLOPS}}{\text{s}} \right\} \\ &= 10.25 \, \frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{5}$$

2. For $x = y$
   Since there is no stencil operation here, the following computations hold true for xSize=2000, xSize=20000 and xSize=400000. In the following Kernel We have;

   - – 1 LD
     – 1 MUL

  - 1 ADD
- This means that we can combine the operations into 1 FMA AVX512. Registers.

Therefore, all operations can be carried out in 1 cycle. Also, note that because we accumulate on dotres, this variable always stays in the register and it does not count as a ST instruction in the kernel. This is because accumulation happens within the register and no additional memory writes are required until after the loop is completed. Thus,

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{\text{cycle}} = \frac{16\ FLOPS}{1\ cycles}$$

and therefore

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 16 \times 18 \times 2.2 = 633.6 \frac{GFLOPS}{s}$$

Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{2}{1 \times 8} = \frac{2}{8} \frac{FLOPS}{Bytes}$$

And we know that the bandwidth $b_s = 82GB/s$. Thus, peak performance for the kernel dotProduct is given by

$$\begin{aligned} P_{x=y}^{\text{dotProduct}} &= \min\left\{P_{\text{1 ccNUMA}}^{\text{max}},\ I \times b_s\right\} \\ &= \min\left\{633.6\ \frac{\text{GFLOPS}}{\text{s}},\ \frac{2 \times 82}{8}\ \frac{\text{GFLOPS}}{\text{s}}\right\} \\ &= 20.5\ \frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{6}$$

1.3.4 (K4) Gs PreCon:

```
//forward substitution
for ( int j=1; j<ySize-1; ++j)
{
    for ( int i=1; i<xSize-1; ++i)
    {
        (*x)(j,i) = w_c*((*rhs)(j,i) + (w_y*(*x)(j-1,i) + w_x*(*x)(j,i-1)));
    }
}
//backward substitution
for ( int j=ySize-2; j>0; --j)
{
    for ( int i=xSize-2; i>0; --i)
    {
        (*x)(j,i) = (*x)(j,i) + w_c*(w_y*(*x)(j+1,i) + w_x*(*x)(j,i+1));
    }
}
```

1. For xSize=2000, xSize=20000

   (a) forward substitution
   Since the layer condition is satisfied, we have

   i. 2 LD
   ii. 1 ST
   iii. 3 MUL
   iv. 2 ADD

   Thus it can be combined into 2FMA and 1MUL and would require 2 cycles,

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{\text{cycle}} = \frac{40\ FLOPS}{2\ cycles} = \frac{20 FLOPS}{cycles}$$

   and therefore

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 20 \times 18 \times 2.2 = 792.0 \frac{GFLOPS}{s}$$

   Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{5}{3 \times 8} = \frac{5}{24} \frac{FLOPS}{Bytes}$$

   And we know that the bandwidth $b_s = 82 GB/s$. Thus, peak performance for the kernel dotProduct is given by

$$\begin{aligned} P_{\text{forward}}^{\text{GSPreCon}} &= \min\left\{P_{\text{1 ccNUMA}}^{\text{max}},\, I \times b_s\right\} \\ &= \min\left\{792.0\,\frac{\text{GFLOPS}}{\text{s}},\, \frac{5 \times 82}{24}\,\frac{\text{GFLOPS}}{\text{s}}\right\} \\ &= 17.08\,\frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{7}$$

   (b) backward substitution
   Since the layer condition is satisfied we have

- 1 LD
- 1 ST
- 3 MUL
- 2 ADD

Thus we have,

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{\text{cycle}} = \frac{40\ FLOPS}{2\ cycles} = \frac{20 FLOPS}{cycles}$$

and therefore

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 20 \times 18 \times 2.2 = 792.0 \frac{GFLOPS}{s}$$

Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{5}{2 \times 8} = \frac{5}{16} \frac{FLOPS}{Bytes}$$

And we know that the bandwidth $b_s = 82 GB/s$. Thus, peak performance for the kernel dotProduct is given by

$$\begin{aligned} P_{\text{backward}}^{\text{GSPreCon}} &= \min\left\{P_{\text{1 ccNUMA}}^{\text{max}},\, I \times b_s\right\} \\ &= \min\left\{792.0\,\frac{\text{GFLOPS}}{\text{s}},\, \frac{5 \times 82}{16}\,\frac{\text{GFLOPS}}{\text{s}}\right\} \\ &= 25.63\,\frac{\text{GFLOPS}}{\text{s}} \end{aligned} \tag{8}$$

2. For xSize=400000

   (a) forward substitution
   Since the layer condition is not satisfied, we have

      i. 3 LD
      ii. 1 ST
      iii. 3 MUL
      iv. 2 ADD

   Thus we have,

$$P_{core}^{max} = \frac{\text{FLOPS per AVX Iteration}}{\text{cycle}} = \frac{40\ FLOPS}{2\ cycles} = \frac{20 FLOPS}{cycles}$$

   and therefore

$$P_{\text{1 ccNUMA}}^{max} = P_{core}^{max} \times n_{threads} \times f_{GHz} = 20 \times 18 \times 2.2 = 792.0 \frac{GFLOPS}{s}$$

   Computational Intensity is given by

$$I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{5}{4 \times 8} = \frac{5}{32} \frac{FLOPS}{Bytes}$$

   And we know that the bandwidth $b_s = 82 GB/s$. Thus, peak performance for the kernel dotProduct is given by

$$
\begin{aligned}
P^{\text{GSPreCon}}_{\text{forward}} &= \min\left\{P^{\text{max}}_{\text{1 ccNUMA}},\, I \times b_s\right\} \\
&= \min\left\{792.0\,\frac{\text{GFLOPS}}{\text{s}},\, \frac{5\times 82}{32}\,\frac{\text{GFLOPS}}{\text{s}}\right\} \\
&= 12.81\,\frac{\text{GFLOPS}}{\text{s}}
\end{aligned} \tag{9}
$$

(b) backward substitution
Since the layer condition is satisfied we have

- 2 LD
- 1 ST
- 3 MUL
- 2 ADD

Thus we have,

$$
P^{max}_{core} = \frac{\text{FLOPS per AVX Iteration}}{\text{cycle}} = \frac{40\ FLOPS}{2\ cycles} = \frac{20 FLOPS}{cycles}
$$

and therefore

$$
P^{max}_{\text{1 ccNUMA}} = P^{max}_{core} \times n_{threads} \times f_{GHz} = 20 \times 18 \times 2.2 = 792.0 \frac{GFLOPS}{s}
$$

Computational Intensity is given by

$$
I = \frac{\text{N FLOPS}}{\text{V Bytes}} = \frac{5}{3\times 8} = \frac{5}{24}\frac{FLOPS}{Bytes}
$$

And we know that the bandwidth $b_s = 82GB/s$. Thus, peak performance for the kernel dotProduct is given by

$$
\begin{aligned}
P^{\text{GSPreCon}}_{\text{backward}} &= \min\left\{P^{\text{max}}_{\text{1 ccNUMA}},\, I \times b_s\right\} \\
&= \min\left\{792.0\,\frac{\text{GFLOPS}}{\text{s}},\, \frac{5\times 82}{24}\,\frac{\text{GFLOPS}}{\text{s}}\right\} \\
&= 17.08\,\frac{\text{GFLOPS}}{\text{s}}
\end{aligned} \tag{10}
$$

1.4. Formulae

To calculate the overall performance we will use the following formulae

$$\boxed{\begin{aligned} T_{total} &= \sum T_{kernel} \\ T_{kernel} &= \frac{Grid\ Size \times FLOPS}{P^{kernel}} \\ P_{total} &= \frac{Iterations}{T_{total}} \end{aligned}}$$

1.5. *roofline* model for CG

```
    while( (iter<niter) && (alpha_0>tol*tol) && (IS_VALID(alpha_0)) )
    {
        pde->applyStencil(v,p);
        lambda =  alpha_0/dotProduct(v,p);
        //Update x
        axpby(x, 1.0, x, lambda, p);
        //Update r
        axpby(r, 1.0, r, -lambda, v);
        alpha_1 = dotProduct(r,r);
        //Update p
        axpby(p, 1.0, r, alpha_1/alpha_0, p);
        alpha_0 = alpha_1;
#ifdef DEBUG
        printf("iter = %d, res = %.15e\n", iter, alpha_0);
#endif
        ++iter;
    }
```

As we can see that the CG solver above uses

- applyStenil(,) kernel one time,
- dotProduct (different) kernel one time,
- dotProduct (same) kernel one time, and
- axpby(, , , ,) kernel three times,

1.5.1 for grid size 2000x20000 (Ny x Nx)

Thus our calculation then becomes

$$\begin{aligned}
T_{\text{total}} &= T_{\text{applyStencil}} + T_{\text{dotProduct}_{\text{same}}} + T_{\text{dotProduct}_{\text{different}}} + 3 \times T_{\text{axpby}} \\
&= 2000 \times 20000 \left[\frac{7FLOPS}{P^{applyStencil}_{xSize=20000}} + \frac{2FLOPS}{P^{dotProduct}_{x=y}} + \frac{2FLOPS}{P^{dotProduct}_{x\neq y}} + \frac{3 \times 3FLOPS}{P^{axpby}} \right] \\
&= 2000 \times 20000 \left[\frac{7FLOPS}{23.92\frac{GFLOPS}{s}} + \frac{2FLOPS}{20.50\frac{GFLOPS}{s}} + \frac{2FLOPS}{10.25\frac{GFLOPS}{s}} + \frac{3 \times 3FLOPS}{10.25\frac{GFLOPS}{s}} \right] \\
&= 2000 \times 20000 \left[\frac{7 \times 10^{-9}s}{23.92} + \frac{2 \times 10^{-9}s}{20.50} + \frac{2 \times 10^{-9}s}{10.25} + \frac{9 \times 10^{-9}s}{10.25} \right] \\
&= 2000 \times 20000 \times 10^{-9} \left[\frac{7}{23.92} + \frac{2}{20.50} + \frac{2}{10.25} + \frac{9}{10.25} \right] s \\
&= 0.04 \times 1.46s \\
&= 0.0584s
\end{aligned}$$

and

$$\begin{aligned}
P^{CG}_{2000\times20000} &= \frac{2000 \times 20000It}{T_{total}} \\
&= \frac{2000 \times 20000}{0.0584} \frac{It}{s} \\
&= 684.93 \frac{MIt}{s} \quad or \quad 684.93 \frac{MLUP}{s}
\end{aligned}$$

1.5.2 for grid size 20000×2000

Here our calculation is same, namely

$$\begin{aligned}
T_{\text{total}} &= T_{\text{applyStencil}} + T_{\text{dotProduct}_{\text{same}}} + T_{\text{dotProduct}_{\text{different}}} + 3 \times T_{\text{axpby}} \\
&= 20000 \times 2000 \left[\frac{7FLOPS}{P^{applyStencil}_{xSize=20000}} + \frac{2FLOPS}{P^{dotProduct}_{x=y}} + \frac{2FLOPS}{P^{dotProduct}_{x\neq y}} + \frac{3 \times 3FLOPS}{P^{axpby}} \right] \\
&= 20000 \times 2000 \left[\frac{7FLOPS}{23.92\frac{GFLOPS}{s}} + \frac{2FLOPS}{20.50\frac{GFLOPS}{s}} + \frac{2FLOPS}{10.25\frac{GFLOPS}{s}} + \frac{3 \times 3FLOPS}{10.25\frac{GFLOPS}{s}} \right] \\
&= 20000 \times 2000 \left[\frac{7 \times 10^{-9}s}{23.92} + \frac{2 \times 10^{-9}s}{20.50} + \frac{2 \times 10^{-9}s}{10.25} + \frac{9 \times 10^{-9}s}{10.25} \right] \\
&= 20000 \times 2000 \times 10^{-9} \left[\frac{7}{23.92} + \frac{2}{20.50} + \frac{2}{10.25} + \frac{9}{10.25} \right] s \\
&= 0.04 \times 1.46s \\
&= 0.0584s
\end{aligned}$$

and

$$\begin{aligned}
P^{CG}_{20000\times2000} &= \frac{20000 \times 2000It}{T_{total}} \\
&= \frac{20000 \times 2000}{0.0584} \frac{It}{s} \\
&= 684.93 \frac{MIt}{s} \quad or \quad 684.93 \frac{MLUP}{s}
\end{aligned}$$

1.5.3 for grid size 1000x40000

Here we have

$$
\begin{aligned}
T_{\text{total}} &= T_{\text{applyStencil}} + T_{\text{dotProduct}_{\text{same}}} + T_{\text{dotProduct}_{\text{different}}} + 3 \times T_{\text{axpby}} \\
&= 1000 \times 400000 \left[\frac{7FLOPS}{P^{applyStencil}_{xSize=400000}} + \frac{2FLOPS}{P^{dotProduct}_{x=y}} + \frac{2FLOPS}{P^{dotProduct}_{x\neq y}} + \frac{3 \times 3FLOPS}{P^{axpby}} \right] \\
&= 1000 \times 400000 \left[\frac{7FLOPS}{14.35 \frac{GFLOPS}{s}} + \frac{2FLOPS}{20.50 \frac{GFLOPS}{s}} + \frac{2FLOPS}{10.25 \frac{GFLOPS}{s}} + \frac{3 \times 3FLOPS}{10.25 \frac{GFLOPS}{s}} \right] \\
&= 1000 \times 400000 \left[\frac{7 \times 10^{-9} s}{14.35} + \frac{2 \times 10^{-9} s}{20.50} + \frac{2 \times 10^{-9} s}{10.25} + \frac{9 \times 10^{-9} s}{10.25} \right] \\
&= 1000 \times 400000 \times 10^{-9} \left[\frac{7}{14.35} + \frac{2}{20.50} + \frac{2}{10.25} + \frac{9}{10.25} \right] s \\
&= 0.4 \times 1.66 s \\
&= 0.664 s
\end{aligned}
$$

and

$$
\begin{aligned}
P^{CG}_{1000 \times 400000} &= \frac{1000 \times 400000 It}{T_{total}} \\
&= \frac{1000 \times 400000}{0.664} \frac{It}{s} \\
&= 602.41 \frac{MIt}{s} \quad or \quad 602.41 \frac{MLUP}{s}
\end{aligned}
$$

1.6. *roofline* model for PCG

```
while( (iter<niter) && (res_norm_sq>tol*tol) && (IS_VALID(res_norm_sq)) )
{
    pde->applyStencil(v,p);
    lambda =  alpha_0/dotProduct(v,p);
    //Update x
    axpby(x, 1.0, x, lambda, p);
    //Update r
    axpby(r, 1.0, r, -lambda, v);
    res_norm_sq = dotProduct(r,r);
    //Update z
    pde->GSPreCon(r, z);
    alpha_1 = dotProduct(r,z);
    //Update p
    axpby(p, 1.0, z, alpha_1/alpha_0, p);
    alpha_0 = alpha_1;

#ifdef DEBUG
    printf("iter = %d, res = %.15e\n", iter, res_norm_sq);
#endif
    ++iter;
}
```

As we can see that the PCG solver above uses

- applyStenil(,) kernel one time,
- dotProduct (different) kernel two time,
- dotProduct (same) kernel one time, and
- axpby(, , , ,) kernel three times,
- GSPreCon (,) one time

In the following computations we compute

$$T_{\text{GSPreCon}} = T_{forward} + T_{backward}$$

*1.6.1 for grid size 2000 * 20000*

$$
\begin{aligned}
T_{\text{total}} &= T_{\text{applyStencil}} + T_{\text{dotProduct}_{\text{same}}} + T_{\text{dotProduct}_{\text{different}}} + 3 \times T_{\text{axpby}} + T_{\text{GSPreCon}} \\
&= 2000 \times 20000 \left[\frac{7FLOPS}{P^{applyStencil}_{xSize=2000}} + \frac{2FLOPS}{P^{dotProduct}_{x=y}} + \frac{2 \times 2FLOPS}{P^{dotProduct}_{x \neq y}} + \frac{3 \times 3FLOPS}{P^{axpby}} \right. \\
&\left. + \frac{2FLOPS}{P^{forward}} + \frac{2FLOPS}{P^{backward}} \right] \\
&= 2000 \times 20000 \left[\frac{7FLOPS}{23.92 \frac{GFLOPS}{s}} + \frac{2FLOPS}{20.50 \frac{GFLOPS}{s}} + \frac{2 \times 2FLOPS}{10.25 \frac{GFLOPS}{s}} + \frac{3 \times 3FLOPS}{10.25 \frac{GFLOPS}{s}} \right. \\
&\left. + \frac{2}{17.08 \frac{GFLOPS}{s}} + \frac{2}{25.63 \frac{GFLOPS}{s}} \right] \\
&= 2000 \times 20000 \left[\frac{7 \times 10^{-9} s}{23.92} + \frac{2 \times 10^{-9} s}{20.50} + \frac{4 \times 10^{-9} s}{10.25} + \frac{9 \times 10^{-9} s}{10.25} \right. \\
&\left. + \frac{2 \times 10^{-9} s}{17.08} + \frac{2 \times 10^{-9} s}{25.63} \right] \\
&= 2000 \times 20000 \times 10^{-9} \left[\frac{7}{23.92} + \frac{2}{20.50} + \frac{4}{10.25} + \frac{9}{10.25} + \frac{2}{17.08} + \frac{2}{25.63} \right] s \\
&= 0.04 \times 1.85 s \\
&= 0.074 s
\end{aligned}
$$

$$
\begin{aligned}
P^{PCG}_{2000 \times 20000} &= \frac{2000 \times 20000 It}{T_{total}} \\
&= \frac{2000 \times 20000}{0.074} \frac{It}{s} \\
&= 539.48 \frac{MIt}{s} \quad or \quad 539.48 \frac{MLUP}{s}
\end{aligned}
$$

*1.6.2 for grid size 20000 * 2000*

$$
\begin{aligned}
T_{\text{total}} &= T_{\text{applyStencil}} + T_{\text{dotProduct}_{\text{same}}} + T_{\text{dotProduct}_{\text{different}}} + 3 \times T_{\text{axpby}} + T_{\text{GSPreCon}} \\
&= 20000 \times 2000 \left[\frac{7FLOPS}{P^{applyStencil}_{xSize=20000}} + \frac{2FLOPS}{P^{dotProduct}_{x=y}} + \frac{2 \times 2FLOPS}{P^{dotProduct}_{x \neq y}} + \frac{3 \times 3FLOPS}{P^{axpby}} \right. \\
&\left. + \frac{2FLOPS}{P^{forward}} + \frac{2FLOPS}{P^{backward}} \right] \\
&= 20000 \times 2000 \left[\frac{7FLOPS}{23.92\frac{GFLOPS}{s}} + \frac{2FLOPS}{20.50\frac{GFLOPS}{s}} + \frac{4FLOPS}{10.25\frac{GFLOPS}{s}} + \frac{3 \times 3FLOPS}{10.25\frac{GFLOPS}{s}} \right. \\
&\left. + \frac{2}{17.08\frac{GFLOPS}{s}} + \frac{2}{25.63\frac{GFLOPS}{s}} \right] \\
&= 20000 \times 2000 \left[\frac{7 \times 10^{-9}s}{23.92} + \frac{2 \times 10^{-9}s}{20.50} + \frac{4 \times 10^{-9}s}{10.25} + \frac{9 \times 10^{-9}s}{10.25} + \frac{2 \times 10^{-9}s}{17.08} + \frac{2 \times 10^{-9}s}{25.63} \right] \\
&= 20000 \times 2000 \times 10^{-9} \left[\frac{7}{23.92} + \frac{2}{20.50} + \frac{4}{10.25} + \frac{9}{10.25} + \frac{2}{17.08} + \frac{2}{25.63} \right] s \\
&= 0.04 \times 1.85s \\
&= 0.074s
\end{aligned}
$$

$$
\begin{aligned}
P^{PCG}_{20000 \times 2000} &= \frac{20000 \times 2000It}{T_{total}} \\
&= \frac{20000 \times 2000}{0.074} \frac{It}{s} \\
&= 539.48 \frac{MIt}{s} \quad or \quad 539.48 \frac{MLUP}{s}
\end{aligned}
$$

*1.6.3 for grid size 1000 * 400000*

$$
\begin{aligned}
T_{\text{total}} &= T_{\text{applyStencil}} + T_{\text{dotProduct}_{\text{same}}} + T_{\text{dotProduct}_{\text{different}}} + 3 \times T_{\text{axpby}} T_{\text{GSPrecon}} \\
&= 1000 \times 400000 \left[\frac{7FLOPS}{P^{applyStencil}_{xSize=400000}} + \frac{2FLOPS}{P^{dotProduct}_{x=y}} + \frac{2 \times 2FLOPS}{P^{dotProduct}_{x \neq y}} + \frac{3 \times 3FLOPS}{P^{axpby}} \right. \\
&\left. + \frac{2FLOPS}{P^{forward}} + \frac{2FLOPS}{P^{backward}} \right] \\
&= 1000 \times 400000 \left[\frac{7FLOPS}{14.35 \frac{GFLOPS}{s}} + \frac{2FLOPS}{20.50 \frac{GFLOPS}{s}} + \frac{2 \times 2FLOPS}{10.25 \frac{GFLOPS}{s}} + \frac{3 \times 3FLOPS}{10.25 \frac{GFLOPS}{s}} \right. \\
&\left. + \frac{2GFLOPS}{12.81 \frac{GFLOPS}{s}} + \frac{2}{17.08 \frac{GFLOPS}{s}} \right] \\
&= 1000 \times 400000 \left[\frac{7 \times 10^{-9} s}{14.35} + \frac{2 \times 10^{-9} s}{20.50} + \frac{4 \times 10^{-9} s}{10.25} + \frac{9 \times 10^{-9} s}{10.25} \right. \\
&\left. + \frac{2 \times 10^{-9} s}{12.81} + \frac{2 \times 10^{-9} s}{17.08} \right] \\
&= 1000 \times 400000 \times 10^{-9} \left[\frac{7}{14.35} + \frac{2}{20.50} + \frac{4}{10.25} + \frac{9}{10.25} + \frac{2}{12.81} + \frac{2}{17.08} \right] s \\
&= 0.4 \times 2.12 s \\
&= 0.85 s
\end{aligned}
$$

$$
\begin{aligned}
P^{PCG}_{1000 \times 400000} &= \frac{1000 \times 400000 It}{T_{total}} \\
&= \frac{1000 \times 400000}{0.85} \frac{It}{s} \\
&= 470.17 \frac{MIt}{s} \quad or \quad 470.17 \frac{MLUP}{s}
\end{aligned}
$$

Grid Size	2000×20000	20000×2000	1000×40000
CG (MLUPS)	684.93	684.93	602.41
PCG (MLUPS)	539.48	539.48	470.17

Table 1: Roofline Performance Results

1.7. Conclusion

The roofline performance results can be summarised as in Table 1. The performance is predicted to drop for the grid size 1000×400000 as it violates the layer condition. Data does not fit completely into the L3 cache and thus during the stencil computations there are more cache misses and thus the data has to be brought more frequently from the main memory to the computation units. As the amount of data required to be called from the main memory increases so the travel time along with the latency, thus increasing the total time resulting into performance drop.

2. Part 2

Check whether you attain the roofline performance by running the code on 1 cc-NUMA domain of Fritz for the three dimensions given above. Timings are already included. Run the code using following command.

To begin the parallelisation of the code we added a header

```
#include <omp.h>
```

to **Grid.cpp** and **Pde.cpp**. Then we parallelised the code in the **applyStencil**, **dotproduct**, **axpby**, **GSPreCon** and other functions from **Grid.cpp** and **PDE.cpp** using

- `#pragma omp parallel`
- `#pragma omp parallel for`
- `schedule(static)`

wherever required. To validate the predictions performance was measured on FRITZ. Firstly, a single interactive node was requested using the command.

```
salloc --nodes=1 --time=02:00:00 -p singlenode -C hwperf
```

2.1. ptfscam.sh

The following jobscript was run by using the command

```
sbatch ptfscam.sh
```

and the results were stored in

```
jobscript_results
```

```
#!/usr/bin/bash -l
#SBATCH --time=01:00:00
#SBATCH --nodes=1
#SBATCH --job-name=ptfscam
#SBATCH --output=jobscript_results/ptfscam.out
#SBATCH --export=NONE
#SBATCH --constraint=hwperf
#SBATCH --partition=singlenode

unset SLURM_EXPORT_ENV

# Load modules
module load intel
module load likwid
LIKWID=on CXX=icpx make

# Define grid sizes
grid_sizes=(
    "2000 20000"
    "20000 2000"
    "1000 400000"
)

for size in "${grid_sizes[@]}"; do
    read Ny Nx <<< "$size"
    # ============================
    # Run with 1ccNUMA (18 cores)
    # ============================

    log_file="jobscript_results/ptfscam_Ny${Ny}-Nx${Nx}-1ccNUMA.txt"
    > "$log_file"
    echo "Running Ny=$Ny, Nx=$Nx, 1ccNUMA" >> "$log_file"

    srun ./test >> "$log_file" 2>&1
    srun --cpu-freq=2200000-2200000:performance likwid-pin -C S0:0-17
    likwid-perfctr -g MEM -m ./perf $Ny $Nx >> "$log_file" 2>&1

    # ============================
    # Run with 4ccNUMA (72 cores)
    # ============================
```

```
    log_file="jobscript_results/ptfscam_Ny${Ny}-Nx${Nx}-4ccNUMA.txt"
    > "$log_file"
    echo "Running Ny=$Ny, Nx=$Nx, 4ccNUMA" >> "$log_file"
    srun ./test >> "$log_file" 2>&1
    srun --cpu-freq=2200000-2200000:performance likwid-pin -C 0-71
    likwid-perfctr -g MEM -m ./perf $Ny $Nx >> "$log_file" 2>&1

done
echo "All runs completed."
```

This jobscript enabled us to

1. Compile the code using

   ```
   LIKWID=on CXX=icpx make
   ```

2. Select the grid sizes (in the form $N_y \times N_x$) of
   (a) 2000×20000
   (b) 20000×2000
   (c) 1000×400000
3. Run the code on 1ccNUMA and 4CCNUMA domains respectively with the CPU frequency of 2.2GHZ

   ```
   --cpu-freq=2200000-2200000:performance
   ```

Grid Size	2000×20000	20000×2000	1000×40000
CG (MLUPS)	630.932739	631.714433	578.034097
PCG (MLUPS)	200.277635	199.666856	118.397238

Table 2: Real Performance Results

2.2. Performance Results

The real performance results can be seen in Table 2. We can see that the performance drops for the grid size 1000×400000 as predicted in the roofline model.

3. Part 3

Measure the code balance in [bytes/LUP] of 'applyStencil' and 'GSPreCon' kernels on 1 ccNUMAdomain (18 threads) for the three grid dimensions and comment whether it agrees with your model. You can use LIKWID for the measurement.

To calculate the code balance we use the following formula

$$B_c = \frac{\text{data volume}}{\text{call count} \times (\text{xSize - 2}) \times (\text{ySize-2})}$$

Region APPLY_STENCIL, Group 1: MEM

Region Info	HWThread 0
RDTSC Runtime [s]	0.607030
call count	45

(a) APPLY STENCIL

Metric	HWThread 0
Runtime (RDTSC) [s]	0.6070
Runtime unhalted [s]	0.5489
Clock [MHz]	2194.7959
CPI	0.3535
Memory read bandwidth [MBytes/s]	47402.4978
Memory read data volume [GBytes]	28.7747
Memory write bandwidth [MBytes/s]	23912.3786
Memory write data volume [GBytes]	14.5155
Memory bandwidth [MBytes/s]	71314.8765
Memory data volume [GBytes]	43.2902

(b) APPLY STENCIL

Region GS_PRE_CON, Group 1: MEM

Region Info	HWThread 0
RDTSC Runtime [s]	2.878122
call count	21

(c) GS PRE CON

Metric	HWThread 0
Runtime (RDTSC) [s]	2.8781
Runtime unhalted [s]	2.6217
Clock [MHz]	2194.7950
CPI	0.9758
Memory read bandwidth [MBytes/s]	7162.6747
Memory read data volume [GBytes]	20.6151
Memory write bandwidth [MBytes/s]	4301.9593
Memory write data volume [GBytes]	12.3816
Memory bandwidth [MBytes/s]	11464.6340
Memory data volume [GBytes]	32.9966

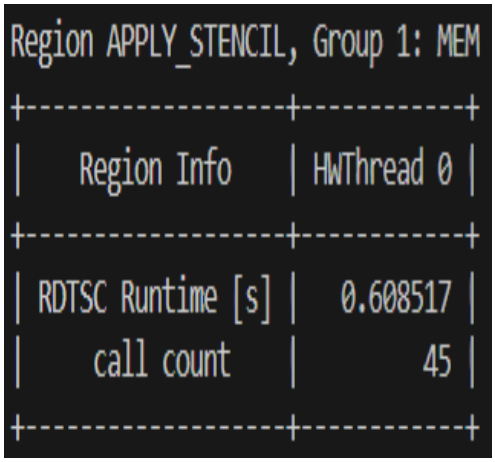
(d) GS PRE CON

Figure 2: srun –cpu-freq=2200000-2200000:performance likwid-pin -C S0:0-17 likwid-perfctr -g MEM -m ./perf 2000 20000

3.1. for grid size 2000 * 20000

3.1.1 applyStencil

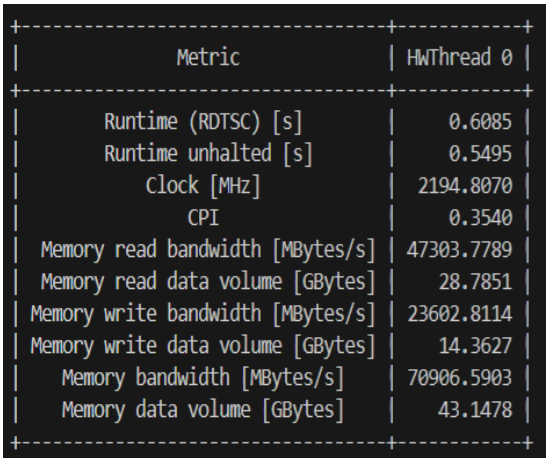
- data volume = 43.2902 GB
- call count = 45
- xSize-2 = 19998



Region APPLY_STENCIL, Group 1: MEM

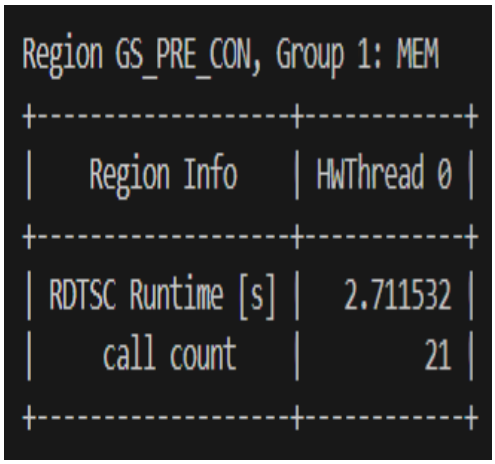
Region Info	HWThread 0
RDTSC Runtime [s]	0.608517
call count	45

(a) APPLY STENCIL



Metric	HWThread 0
Runtime (RDTSC) [s]	0.6085
Runtime unhalted [s]	0.5495
Clock [MHz]	2194.8070
CPI	0.3540
Memory read bandwidth [MBytes/s]	47303.7789
Memory read data volume [GBytes]	28.7851
Memory write bandwidth [MBytes/s]	23602.8114
Memory write data volume [GBytes]	14.3627
Memory bandwidth [MBytes/s]	70906.5903
Memory data volume [GBytes]	43.1478

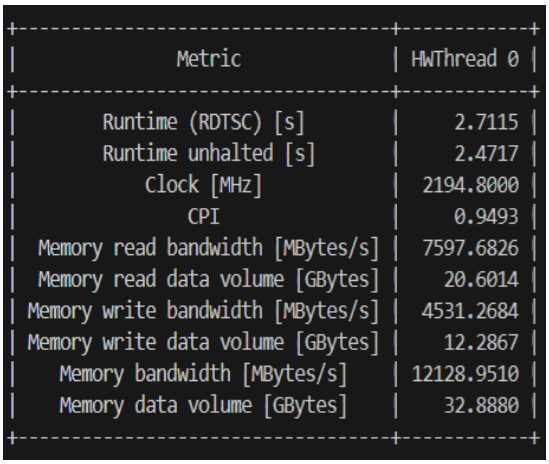
(b) APPLY STENCIL



Region GS_PRE_CON, Group 1: MEM

Region Info	HWThread 0
RDTSC Runtime [s]	2.711532
call count	21

(c) GS PRE CON



Metric	HWThread 0
Runtime (RDTSC) [s]	2.7115
Runtime unhalted [s]	2.4717
Clock [MHz]	2194.8000
CPI	0.9493
Memory read bandwidth [MBytes/s]	7597.6826
Memory read data volume [GBytes]	20.6014
Memory write bandwidth [MBytes/s]	4531.2684
Memory write data volume [GBytes]	12.2867
Memory bandwidth [MBytes/s]	12128.9510
Memory data volume [GBytes]	32.8880

(d) GS PRE CON

Figure 3: srun –cpu-freq=2200000-2200000:performance likwid-pin -C S0:0-17 likwid-perfctr -g MEM -m ./perf 20000 2000

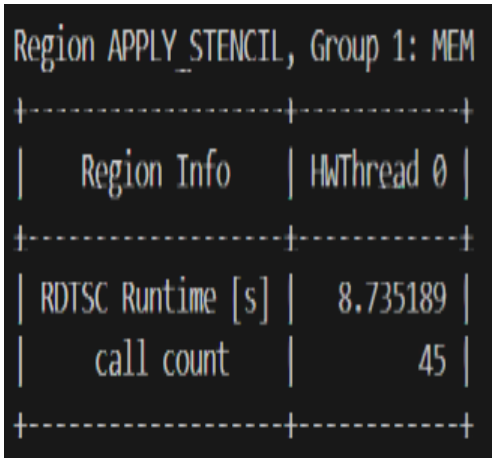
- ySize-2 = 1998
- $B_c = 24.08 \frac{B}{LUP}$

Comparing this with our computations of $24 \frac{B}{LUP}$ is nearly accurate.

3.1.2 GSPrecon

- data volume = 32.9966 GB
- call count = 21
- xSize-2 = 19998
- ySize-2 = 1998
- $B_c = 39.32 \frac{B}{LUP}$

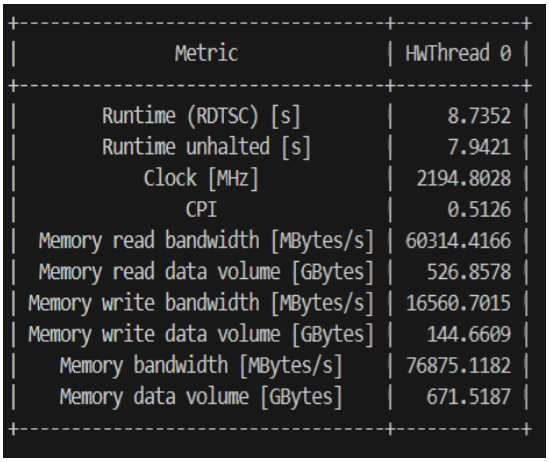
Comparing this with our calculation of $40 \frac{B}{LUP}$ is nearly accurate.



Region APPLY_STENCIL, Group 1: MEM

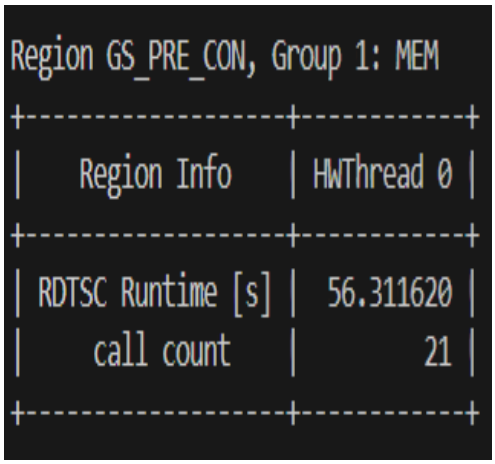
Region Info	HWThread 0
RDTSC Runtime [s]	8.735189
call count	45

(a) APPLY STENCIL



Metric	HWThread 0
Runtime (RDTSC) [s]	8.7352
Runtime unhalted [s]	7.9421
Clock [MHz]	2194.8028
CPI	0.5126
Memory read bandwidth [MBytes/s]	60314.4166
Memory read data volume [GBytes]	526.8578
Memory write bandwidth [MBytes/s]	16560.7015
Memory write data volume [GBytes]	144.6609
Memory bandwidth [MBytes/s]	76875.1182
Memory data volume [GBytes]	671.5187

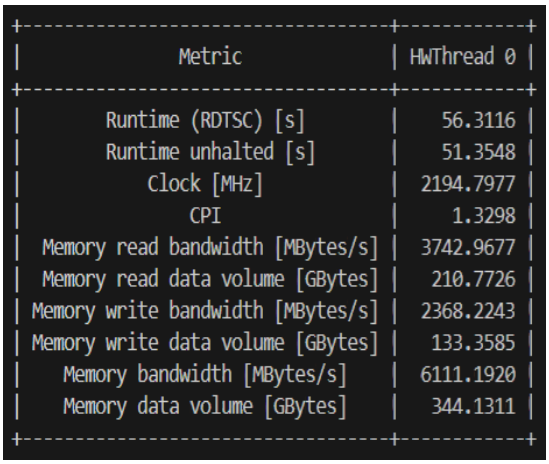
(b) APPLY STENCIL



Region GS_PRE_CON, Group 1: MEM

Region Info	HWThread 0
RDTSC Runtime [s]	56.311620
call count	21

(c) GS PRE CON



Metric	HWThread 0
Runtime (RDTSC) [s]	56.3116
Runtime unhalted [s]	51.3548
Clock [MHz]	2194.7977
CPI	1.3298
Memory read bandwidth [MBytes/s]	3742.9677
Memory read data volume [GBytes]	210.7726
Memory write bandwidth [MBytes/s]	2368.2243
Memory write data volume [GBytes]	133.3585
Memory bandwidth [MBytes/s]	6111.1920
Memory data volume [GBytes]	344.1311

(d) GS PRE CON

Figure 4: srun –cpu-freq=2200000-2200000:performance likwid-pin -C S0:0-17 likwid-perfctr -g MEM -m ./perf 1000 400000

3.2. for grid size 20000 * 2000

3.2.1 applyStencil

- data volume = 43.1478 GB
- call count = 45
- ySize-2 = 19998
- xSize-2 = 1998
- $B_c = 24\frac{B}{LUP}$

Comparing this with our computations of $24\frac{B}{LUP}$ is equal.

3.2.2 GSPrecon

- data volume = 32.8880 GB
- call count = 21

- ySize-2 = 19998
- xSize-2 = 1998
- $B_c = 39.20 \frac{B}{LUP}$

Comparing this with our calculation of $40 \frac{B}{LUP}$ is nearly accurate.

3.3. for grid size 1000 * 400000

3.3.1 applyStencil

- data volume = 671.5187 GB
- call count = 45
- ySize-2 = 998
- xSize-2 = 399998
- $B_c = 42.05 \frac{B}{LUP}$

Comparing this with our result of $40 \frac{B}{LUP}$ is nearly accurate.

3.3.2 GSPrecon

- data volume = 344.1311 GB
- call count = 21
- ySize-2 = 998
- xSize-2 = 399998
- $B_c = 41.05 \frac{B}{LUP}$

Comparing this with our calculation of $56 \frac{B}{LUP}$ is way far away.

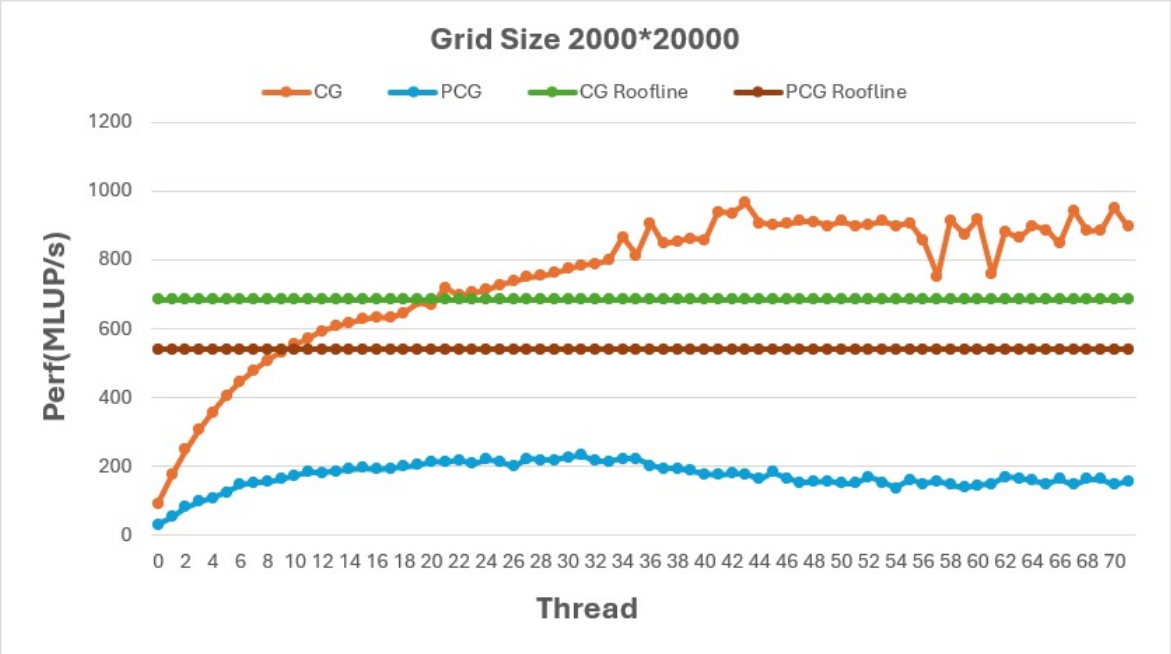


Figure 5: Grid Size 2000*20000

4. Part 4

Plot the scalability of CG and PCG from 1 → 4 ccNUMA domains (4 ccNUMA domains = 1 node = 72 cores) of Fritz. Does it scale perfectly? If not, why? Can you fix it?

For CG, after a certain number of threads the performance saturates and does not improve with additional threads. This indicates a transition to a memory-bandwidth-limited regime, where increasing parallelism no longer increases the achievable throughput. For PCG, this bandwidth-limited regime is reached much earlier due to its higher memory traffic and lower arithmetic intensity.

Performance may be improved by applying cache-aware optimizations such as tiling, which reduce main-memory traffic by improving data locality. This increases the effective arithmetic intensity of the kernel and allows performance to approach higher cache-level roofline limits.

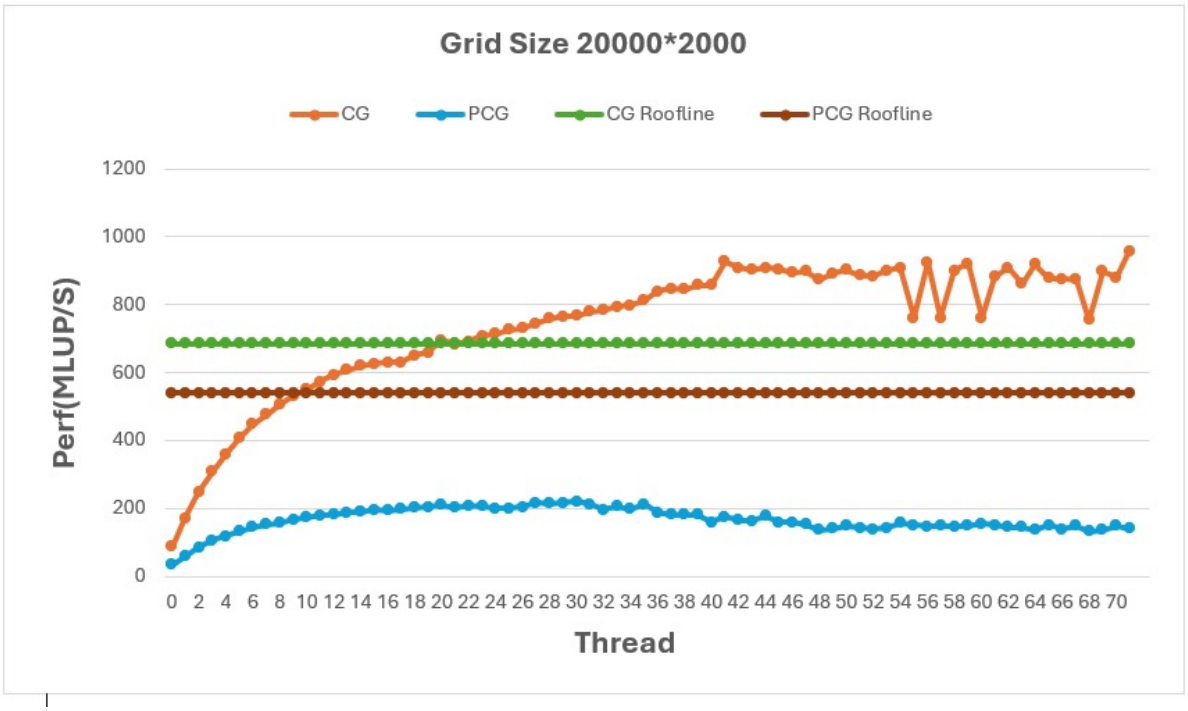


Figure 6: Grid Size 20000 * 2000

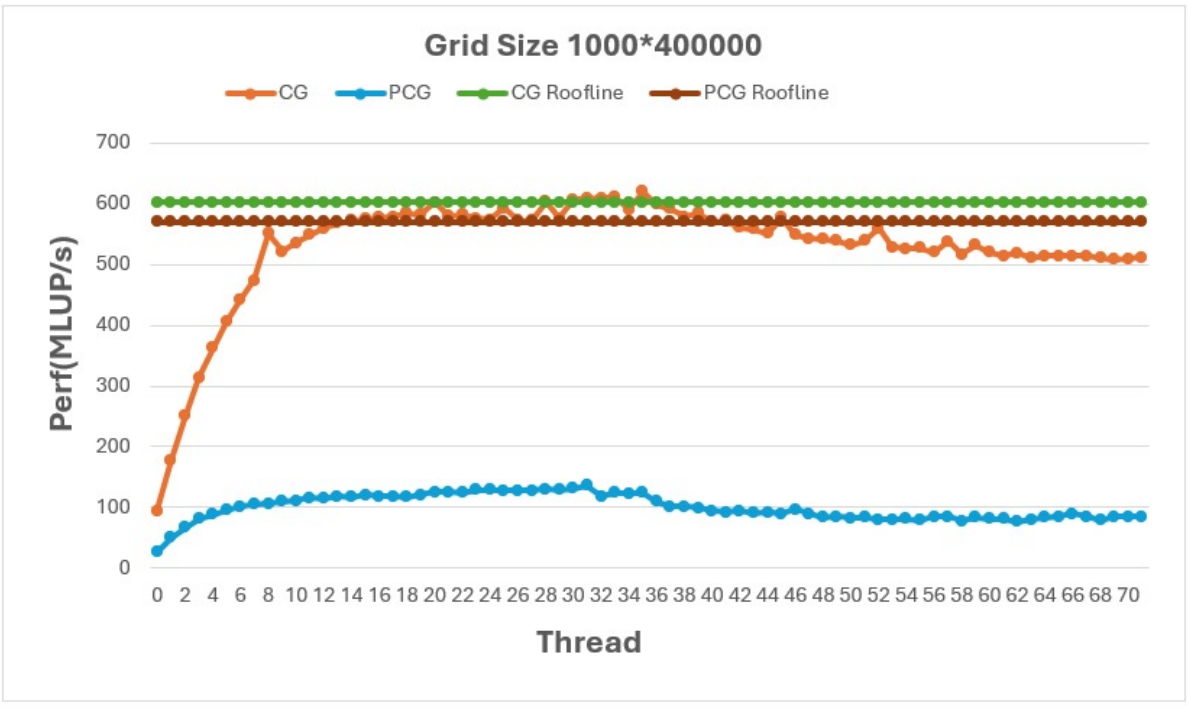


Figure 7: Grid Size 1000 * 40000